



UNIVERSITY OF SALERNO

Computer Science Department

Master Degree

GRADUATION THESIS

Investigating the Role of Unit Tests in Performance Regression Testing Optimization

SUPERVISOR

Prof. Dario Di Nucci

Dr. Antonio Trovato

University of Salerno

CANDIDATE

Daniele Fabiano

ID Number: 0522501738

2024-2025 Academic Year

This thesis was carried out in the



Abstract

Regression testing is a crucial activity to verify software correctness after changes are introduced into the system. This type of activity is expensive, considering that all test cases should be re-executed again. Regression Test Suite Optimization (RTSO) algorithms use techniques, such as Test Case Selection (TCS), to select an appropriate subset of test cases, that has the same coverage of the starting test cases with a reduced cost. The algorithms are mainly used for functional testing, and little consideration is given to their use for non-functional testing, which is even more expensive, especially for verifying system performance. This thesis aims to explore TCS techniques in the context of performance testing, to understand whether unit tests and testing metrics, such as coverage and execution costs, can be reused to define a subset of microbenchmarks and conduct a successful performance regression testing campaign. Two Apache Open Source Systems (Avro and Hive) have been chosen to extract statement coverage data and execution times of the different tests, leveraging JaCoCo, JUnit, and JMH, from two particular pair of commits with known performance issues. The data have been processed in the form of the input for some RTSO algorithms (Additional Greedy, DIV-GA, IGDec-QAOA, QAOA-TCS) to obtain as output a Pareto Optimal Set containing the subsets of the regression test suite, depending on if the approached input data came from a starting unit tests suite or a microbenchmarks suite. The results show how DIV-GA executed on the data from the microbenchmarks suite, is the most effective approach, able to select subsets of test cases with the highest coverage. The quantum algorithms executed on the data from the unit tests suite, are the most efficient ones, able to select subsets of test cases with the lowest execution costs. JMH remains a reliable tool for precise time measurement, with the possibility of converting existing JUnit suites to JMH ones with dedicated tools and further try to reduce testing activities costs, in cases where microbenchmark suites do not exist in the system under consideration. Given the ease of use of DIV-GA and how its execution on the data from the microbenchmarks suite returns well balanced solutions, this approach can be considered the best between the analyzed ones. The final remainder is to choose an approach that suites most the metrics to prioritize and the the system to be tested.

Contents

List of Figures	iv
List of Tables	v
1 Introduction	1
1.1 Application Context	1
1.2 Motivations and Goals	2
1.3 Achieved Results	3
1.4 Thesis Structure	5
2 Background and Related Work	6
2.1 Performance Testing	6
2.1.1 Microbenchmarking	7
2.1.2 Java Microbenchmark Harness	7
2.2 Regression Test Suite Optimization	10
2.3 Traditional Algorithms for RTSO	11
2.4 A Quantum Approach	12
2.4.1 Quantum Computing	12
2.4.2 Quantum Algorithms for RTSO	14
2.5 Related Work	15

3	An Upstream and a Downstream Method to Create and Select Microbenchmarks	17
3.1	Method Overview	17
3.1.1	Upstream Method Approach	17
3.1.2	Downstream Method Approach	18
3.2	Computing Testing Metrics	19
3.2.1	Statement Coverage	19
3.2.2	Execution Costs	20
3.3	Data Preparation	21
3.4	Generating Unit Tests and Microbenchmarks	24
3.4.1	Prompt Engineering	25
3.4.2	Ablation Study	27
3.5	Employed Algorithms	27
3.5.1	Adapting QUBO	28
4	Empirical Study	30
4.1	Goal and Research Questions	30
4.2	Systems Overview	31
4.3	Experiment Configuration	32
4.4	Evaluation Metrics	33
4.5	RQ1 - Which is the best algorithm for selecting a subset of microbenchmarks?	34
4.6	RQ2 - Which is the best approach for creating and selecting microbenchmarks?	38
5	Discussion	40
6	Threats to Validity	42
6.1	Construct Validity	42
6.2	Internal Validity	43
6.3	External Validity	43
6.4	Conclusion Validity	43

7 Conclusions	44
----------------------	-----------

Bibliography	45
---------------------	-----------

List of Figures

2.1	A benchmark example class with JMH	9
3.1	Upstream Method Approach - Building Blocks	18
3.2	Downstream Method Approach - Building Blocks	18
3.3	Source file coverage codified in an XML element	21
3.4	Coverage matrix representation in a JSON file format	22
3.5	Unit tests execution times in an XML file format	22
3.6	Microbenchmark execution times in a JSON file format	23
3.7	Time matrix representation in a JSON file format	23
3.8	Meta-prompt to obtain the prompt for unit tests generation	25
3.9	Meta-prompt to obtain the prompt for microbenchmarks generation	26

List of Tables

2.1	Benchmark Modes of JMH	8
3.1	Generated unit tests and microbenchmarks for each ablation	27
4.1	Lines of code with known performance issues - Avro	31
4.2	Lines of code with known performance issues - Hive	32
4.3	Mean and Standard Deviation for pareto fronts - statement coverage	34
4.4	Dunn's test and $\hat{\Delta}$ effect size - statement coverage	35
4.5	Mean and Standard Deviation for pareto fronts - execution costs (in milliseconds)	36
4.6	Dunn's test and $\hat{\Delta}$ effect size - execution costs (in milliseconds)	37
4.7	Dunn's test and $\hat{\Delta}$ effect size - extra comparisons for execution costs	38

CHAPTER 1

Introduction

1.1 Application Context

Software quality is a huge aspect in the modern software development landscape. The ever-growing user base and rapid evolution of software systems requires that the developers have to respond with best practices, not only to guarantee that the code remains clean, readable, documented and maintainable but also that the final product adheres to standard quality levels. There is therefore the need to increase software quality assurance activities, in order to raise the user experience as its best. Functional testing is the main subset of activities that supports quality assurance, with controlled executions of system components given specific inputs to verify their correctness. Non-functional testing activities are also growing, with the aim to verify other quality attributes not directly related to correctness, such as usability, security and performance. These type of tests typically require the system to be executed in its entirety and under specific conditions. Testing activities are generally time-consuming, particularly, performance testing becomes rapidly impractical by pushing the system under test (SUT) to its limits. Since time is a limited resource, there is also the need to optimize the allocated time for the quality assurance activities.

1.2 Motivations and Goals

When changes are implemented, the testing phase should be executed again to ensure that the system continues to work as expected. Regression Testing is the set of activities that ensure that new introduced changes into a system, have not altered the correctness of already implemented features. The default strategy, e.g. re-test all, consists of executing again all the test cases implemented to find possible regressions, but is the most expensive one and should be discarded in favor of other strategies. Regression Test Suite Optimization (RTSO) comprises a series of techniques in order to reduce the costs of regression testing. One of these techniques is Test Case Selection and consists in selecting a subset of the original test suite to test the modified parts of the system. This technique has been extensively explored for functional testing, but no studies have been identified that replicate its application for performance testing. This thesis propose the application of TCS for performance testing, with two different approaches that differ on what type of starting test suite is used. The upstream approach involves running optimization algorithms on the test suite consisting of unit tests to select a subset of it to be converted in a microbenchmark one. The downstream approach relies directly on what are considered the counterpart of unit tests for performance testing, the microbenchmarks, executing the optimization algorithms on the microbenchmark suite and selecting a subset of it. Four algorithms that represent the State Of The Art (SOTA), have been identified to be used for each approach. The first two are Additional-Greedy [1] and DIV-GA [2], respectively based on greedy and meta-heuristics. The other two are IGDDec-QAOA [3] and QAOA-TCS [4], based on quantum computing. The algorithms are applied on the microbenchmark suite data retrieved from two Apache Open Source Systems¹, namely Avro, a data serialization system, and Hive, a data warehouse, on specific versions by the analyzed pair of commits on the git repository, that have known code fragments with performance issues. The final goal is to compare the various algorithms on the two approaches and understand which one performs better, in terms of effectiveness, considering the statement coverage achieved on the code fragments by the chosen subset of the suite and in terms of efficacy, considering the overall execution costs for the subset of the

¹[Apache Project List - Java Language](#) (Accessed on: 13-01-2026)

suite. The different configurations of the approaches are then compared to establish the better one. For more detailed information, feel free to check out Chapters 2 and 3.

1.3 Achieved Results

After all the output were collected from the two approaches using each algorithm, the mean values and standard deviation values were extracted. For the effectiveness, on the Avro data, DIV-GA is able to select suites with 99% of statement coverage for JUnit and a perfect 100% for JMH suites, which was also confirmed by the Vargha-Delaney $\hat{A}$ effect size[5], where DIV-GA outperformed Add. Greedy and the two quantum algorithms with a large magnitude. On Hive data, similar results were recorded for DIV-GA, but with a lower percentage range of statement coverage from 64% on JUnit suites, to 81% for JMH suites, which still outperformed the other algorithms with a large magnitude. In this case the coverage could be lowered also because Hive had more lines affected by performance issues than Avro. These results tell us that in this particular context, the traditional algorithms are still useful and that the quantum algorithms were inadequate and not ready to solve this problem, rarely reaching more than 50% of target statement coverage, only in one case of QAOA-TCS with 62% on JMH suites for Hive data, which was also the only case where a quantum algorithm beat a traditional algorithm (Additional Greedy). Also, for a comparison between the test tools, JMH achieved better coverage than JUnit, even if the former is not directly suited for white-box testing. This result could be explained by how the two suites were written, mainly relying on existing unit tests on JUnit side, but generating specific microbenchmarks for the code fragments to be checked on JMH side. Talking about efficiency, on the Avro data, DIV-GA is on par with QAOA-TCS, selecting suites with execution times of 10ms on JUnit, but placed third for JMH suites with 25ms, after IGDec-QAOA with 19ms and QAOA-TCS with 16ms. This results were partially confirmed by $\hat{A}$ effect size, where on JUnit side DIV-GA outperformed with large magnitude Add. Greedy and IGDec-QAOA but no sufficient significance was recorded for comparison with QAOA-TCS. On the JMH side, QAOA-TCS outperformed with large magnitude Add. Greedy, that was also outperformed by DIV-GA, but no significant tests were recorded with DIV-GA.

On Hive data, IGDec-QAOA gave best results, selecting suites with execution times of 16ms on JUnit and 41 ms on JMH. The tests also showed how IGDec-QAOA outperformed with large magnitude all other algorithms, except for a no significance comparison with QAOA-TCS on JMH suites. DIV-GA still remained the best solution for traditional algorithms. JMH could record more precise times thanks also to a built-in multiple execution support. JUnit has a built-in support to a single execution and a time recording capability only up to three decimal places, with various cases where a default execution time was necessary to replace the test cases with zero time (0.000) to try reduce the alteration of the statistics. Overall, even if DIV-GA was mostly lacking behind quantum algorithms for the efficiency, except for Avro data on JUnit suites and always above Additional Greedy, can be a balanced solution thanks to an easier access in using traditional algorithms. For the tool comparison, the JUnit execution times appear to be unstable due to the aforementioned issues and JMH should be preferred for its reliable measurement, but the creation of dedicated microbenchmarks would require time for their implementation, increasing the overall testing activities completion time. An interesting compromise could still be achievable through starting JUnit suites, thanks to conversion tools (such as JU2JMH[6]) that can automatically wrap JUnit suites in JMH ones, but with a strong remainder that the microbenchmarks executions have to still consider an overhead, due to the time needed for the conversion and the nature of how the wrapping technique works, increasing the execution of microbenchmarks that actually call the unit tests under the hood. It is also important to remember that the best approach with the best choice of selected algorithm and selected tool, depends also on the type of software that have to be tested and which of the two testing metrics have to be prioritized. For more detailed information, feel free to check out Chapters 4 and 5.

1.4 Thesis Structure

The remainder of the thesis is structured into the following chapters:

- **Chapter 2 - Background:** In this chapter, fundamental concepts are described and already existent approaches are presented, emphasizing how this thesis differs from them with a new provided contribution;
- **Chapter 3 - An Upstream and a Downstream Method to Generate and Select Microbenchmarks:** In this chapter, the two method approaches are presented and all the steps to apply the new approaches are detailed;
- **Chapter 4 - Empirical Study:** In this chapter, the two approaches are statistically compared and quantitatively analyzed to retrieve the necessary information, in order to establish when an approach is better than another;
- **Chapter 5 - Discussion:** In this chapter, there is a qualitative analysis related to the context of analyzed systems and why the approaches yielded those results;
- **Chapter 6 - Threats to Validity:** In this chapter, the limitation of the experimental environment are discussed, with the relative mitigations applied to counteracts the threats to validity;
- **Chapter 7 - Conclusions:** The final chapter, where a brief of all the thesis is presented and some future works are discussed.

CHAPTER 2

Background and Related Work

In this chapter, fundamental concepts are introduced, with a description of the algorithms that will be used and the main tool for microbenchmarking. Related work are analyzed and compared with this thesis.

2.1 Performance Testing

Testing is the phase of the development life-cycle where the goal is to assure the quality attributes of a software. The most important one is the correctness, to guarantee that the software behaviors follow the defined specifications during the previous phases of requirements elicitation and analysis. A software must also meet non-functional requirements in order to enhance the user experience and to do that, there is the necessity to simulate real world scenarios. Software systems are being used by a lot of users at the same time and they have to be realized in a way to guarantee that everyone can have the same type and the same level of experience, reducing delays and improving responsiveness. Among the non-functional requirements, performance is one of the most influential to be addressed [7]. Performance Testing is the set of activities that evaluates software performance attributes, such as responsiveness, scalability, and stability, to find performance regressions.

2.1.1 Microbenchmarking

The performance evaluations are based on the concept of understanding the system performance on different versions and configurations. This type of activity falls under the name of what is called **benchmarking**, a widely method that can be used to compare different tools of the same domain, evaluating and comparing different configurations of the same tool or finding out how a single tool performs under different inputs [8]. In the context of software performance testing, the activities typically deal with an entire system, introducing different inputs through the simulation of workloads in a production-like environment, to compare the system with a fixed performance goal or with different versions of the same system. These characteristics make activities complicated and time-consuming, causing them to not be suitable to be automated and to be executed in small amounts of time. With the advent of Continuous Integration (CI) and Continuous Delivery (CD) practices, a software can be built and tested on every commit and push, emphasizing shorter cycle time and faster delivery to the customers [9]. Traditional functional testing such as unit testing and integration testing can be easily executed automatically during the CI build. Therefore it has to be find a way to have the counterpart of unit tests to monitor the performance of smaller software components on every commit and push, in order to facilitate the detection of performance regressions. **Microbenchmarking** is a technique that aims to solve this issue introducing the so called microbenchmarks, unit tests like performance testing to measure fine-grained performance metrics, such as method-level execution times, throughput, or heap utilization [10]. Microbenchmarks can be then implemented with a specific tool for the used programming language of the software system that has to be tested.

2.1.2 Java Microbenchmark Harness

Java MicroBenchmark Harness (JMH)¹ is a toolkit created to write microbenchmarks and gives the developer the ability to automate the verification of the system performance. It is part of the OpenJDK and considered a de-facto standard. JMH offers an annotation system and a Command Line Interface that can be used for

¹Java Microbenchmark Harness (JMH) - Github Repository (Accessed on: 15-01-26)

benchmarking a method in different ways. To enable a method of a benchmark class, the first annotation to use is **@Benchmark**, that gives the compiler the hint to run the method as a benchmark one. The benchmark mode is specified with the **@BenchmarkMode (Mode)** annotation²:

Table 2.1: Benchmark Modes of JMH

Mode	Usage
Throughput	Measures the number of times per second the benchmark method could be executed
Average Time	Measures the average time it takes for the benchmark method to complete an execution
Sample Time	Measures how long time it takes for the benchmark method to complete an execution, with max, min time etc.
Single Shot Time	Measures how long time a single benchmark method execution takes to complete a run (no Warmup phase)
All	Measures with all the above modes

The measurement has to be set on a time unit with **@OutputTimeUnit (TimeUnit)** annotation. The time units varies from multiples (MINUTES, HOURS, DAYS) of SECONDS and its submultiples (MILLI, NANO, MICRO). Before obtaining a precise measurement, the execution of the benchmark has to be iterated a fixed number of times to satisfy the "Warmup phase", in which the Java Virtual Machine (JVM) applies dynamic compilation techniques and all possible code optimizations. Then the benchmarks reach the "Steady state" and the execution time stabilizes to obtain reliable measurements [11]. The **@Warmup(iterations = n)** annotation specifies the number of warmup iterations where the benchmark is executed but not measured. The number of real iterations is specified with **@Measurement(iterations = m)** annotation, where the benchmark is executed (after the warmups) and measured. It is possible to specify the number of full-runs (warmups and measurements) with the **@Fork (N)** annotation and how many of them are concurrently launched with the **@Thread (M)** annotation. If the Fork value is greater than one, multiple JVM process are sequentially launched and then the results are aggregated. The annotations can be specified for an entire benchmark class or for each benchmark method. It is also possible to set some global parameters and overwrite the ones at method or class-level (e.g. all the microbenchmarks should do the same amount of warmups) through the CLI (e.g. `java -jar target/benchmarks.jar -wi 10`) The code to be benchmarked may need some variables that should not be considered for the

²Jenkov tutorial on JMH (Accessed on: 15-01-26)

measurement. With a State inner Class, it is possible to specify variables used by the code to be benchmarked. The State Class should be annotated with `@State (Scope)`, specifying the Scope that can be for a Thread, a Group of threads or for the entire Benchmark run. A State Class should include methods to initialize a State Object and to dispose it, using the respective `@Setup (Level)` and `@TearDown (Level)` specifying when these methods should be called, such as for a Trial of a benchmark (full-run), for a single Iteration or Invocation of the benchmark method. JVM may also apply other type of optimizations to skip the execution of some benchmarks methods, this can be mitigated thanks to JMH. A computation can be considered "dead" when a benchmark method does not return a value and JVM will remove its execution. To avoid this limitation, the calculated value can be passed to a **Blackhole**, with the method `blackhole.consume()`. A calculation based on constants will often results in the same results, causing the JVM to skip the execution and directly set the result of the calculation. To avoid this behavior, the input of the calculations should come from the variables defined inside the State Class.

```

1 package com.example.jmh
2
3 import org.openjdk.jmh.annotations.*;
4 import org.openjdk.jmh.infra.Blackhole;
5 import java.util.concurrent.TimeUnit;
6
7 @Benchmark
8 @BenchmarkMode(Mode.SingleShotTime)
9 @OutputTimeUnit(TimeUnit.SECONDS)
10 public class CalculatorBenchmark {
11
12     @State(Scope.Thread)
13     public static class MyState {
14
15         @Setup(Level.Trial)
16         public void doSetup() {
17             calculator = new Calculator();
18         }
19
20         public int a = 3;
21         public int b = 5;
22         public Calculator calculator;
23     }
24
25     @Fork(2)
26     @Warmup(iterations = 5)
27     @Measurement(iterations = 10)
28     public void benchSum(MyState state, Blackhole blackhole) {
29         blackhole.consume(calculator.sum(state.a, state.b));
30     }
31
32 }

```

Figure 2.1: A benchmark example class with JMH

2.2 Regression Test Suite Optimization

Software is continuously evolving due to the rapidly change of requirements from the customers and the ability to easy push new releases thanks to modern infrastructures. When there is a new version of a software, the changes introduced to implement the new functionalities, could interfere with the correctness of the already existing ones, it is then necessary to also test them again. Regression testing is a testing activity that involves the re-execution of existing test cases to ensure that the new changes have not altered the already implemented functionalities that were considered corrected. Regression testing helps to find the introduction of new issues in the system, mainly for the functional side, but on a bigger picture this type of activity could also be applied to find issues on other quality attributes of the system, such as performance. The typical way to do regression testing is to re-test all and execute again the existing test cases. This method can be very expensive as it has downsides related to the duration of the new testing campaign and on the resources used for all the test cases. There is the necessity to reduce the effort required for a regression testing campaign through the optimization of the size and/or the order of the test cases contained in a test suite. There are three main techniques that can solve the problem of **Regression Test Suite Optimization (RTSO)** [12]:

Test Suite Minimization (TSM) aims to identify and remove the redundant and obsolete test cases from the test suite. Formally: Let T be the initial test suite, R the set of requirements to satisfy and T' the identified subset of T that can still satisfy R ;

Test Case Selection (TCS) aims to select a subset of test cases that will be used to test the changed parts of a software. Formally: Let P be the initial program to test and P' the modified program to test, T the initial test suite for P' and T' the selected subset of T to test P' ;

Test Case Prioritization (TCP) aims to identify the ideal ordering of test cases to maximize test properties, such as fault detection. Formally: Let T be the initial test suite, PT the set of all possible permutations of T and T' the identified permutation that fits test properties more than any other permutation in PT .

These three techniques can also be combined and form the basis for the implementation of algorithms that can approach the problem of test cases optimization.

2.3 Traditional Algorithms for RTSO

TCS is the more direct technique to find a reduced subset of test cases and build a new test suite. The selection can be done in different ways and there are various algorithms to use. The initial propose made by Rothermel and Harrold [13] was based on a single-objective greedy algorithm. Multi-objective algorithms were also adopted and introduced the concept of Pareto Optimality. A solution X is said to be part of the Pareto Optimal set if it is not dominated by any other solution Y , in a way that the values of the objective vector from the objective functions computed for X , are all better or equal than the ones for Y and there is one value of X that is strictly better. This also means that any Y solution cannot improve one of its objective vector values, without worsening another one and still not surpass X . Yoo and Harman [1] proposed a two-objective additional greedy algorithm considering the coverage and execution cost. The algorithm was also compared with other solutions based on meta-heuristic evolutionary algorithms, such as NSGA-II [14] and his variant vNSGA-II [1]. This comparison showed how greedy based algorithm are not very suitable to produce Pareto Optimal solutions. In greedy algorithms, such as an additional-greedy based algorithm, the multi-objective is implemented in a multi-weight way, combining multiple objectives into a single function by assigning each one a weight. In this way, the greedy based algorithm only produce one solution, that is post-processed to simulate the building of a Pareto Optimal set, by selecting all the non-dominated subsets of the obtained solution, which are built incrementally. These considerations open the doors to new meta-heuristic algorithms ideas that could better explore the search space. Panichella et al. realized DIV-GA [2], which is built on top of NSGA-II and proposes a new method that exploits the evolutionary mechanism introducing diversity directly during the generation of a solution, mitigating genetic drift and premature convergence, considering also a third objective as the past fault coverage of the chosen test case. This algorithm performs better than the previous ones, thanks also to how meta-heuristic algorithms works. In meta-heuristic algorithms, such

as a genetic based algorithm, the multi-objective is implemented through multiple objective functions that give different objective vectors to compare and to natively build a Pareto Optimal set, having already multiple valid solutions. However, as stated by Panichella et al.[2], there is no a clear winner between single-objective and multi-objective algorithms and their combination is not always useful to achieve better results, remarking how issues remains for the resolution of these problem with these type of algorithms. Other issues are also linked to the limitations of a classical and deterministic computation system and to the natural difficulty of resolution, that could be overcome with a new one that is emerging in last years.

2.4 A Quantum Approach

With the advent of Quantum Computing, new resolution and decomposition techniques emerged to create new types of algorithms for this optimization problem. After a brief description of description of the new computation system, RTSO algorithms built on top of two different paradigms are presented.

2.4.1 Quantum Computing

Quantum Computing is a new computation system that can enhance the resolution of problems, using a different way of representing the information, with the counterpart of bits in quantum world. Quantum bits, or qubits, can represent a state that is expressed in the following way:

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$$

where α and β represents the probabilities weights that the qubit can be in either 0 or 1 state. When both weights are bigger than zero, the qubit is in an "intermediate" state where both states have a value, and it is called *superposition*. The state then collapse to the actual value in bit during the phase of measurement, where the actual values of the solution are retrieved. Due to multi-objective RTSO being a NP-Complete problem [15], opens up to the use of non deterministic computation system, to try to speed up the search of a sub-optimal solution. Exploiting the quantum computation

realizing dedicated algorithm for RTSO, gives then the possibility to better explore the solution space. There are two leading quantum paradigms with related hardware:

Adiabatic Quantum Computatioin (AQC) initializes many-qubit quantum state as the ground state of a simple Hamiltonian function, then applying an adiabatic time evolution cause the system to change to a final Hamiltonian function whose ground state represent the solution of the problem. D-Wave realized dedicated machines for this paradigm, that requires an in-purpose hardware where the algorithms are implemented in a more hardcoded and "analog" way. Quantum Annealing (QA) algorithms are executed on this type of computer³;

Gate Model Quantum Computation (GMQC) applies a sequence of quantum gates, the counterpart of logical gates, to a set of quantum bits (qubits), whose states can be measured at the end of the computation to retrieve the problem solution. Multiple quantum gates can be applied, realizing a quantum circuit. IBM⁴ realized dedicated machines for this paradigm, which are designed to be universal and general purpose, implementing the algorithms through the programming of the gates in a more digital way. Quantum Approximate Optimization Algorithms (QAOA) are executed on this type of computers.

The paradigms are considered to be polynomially equivalent in a way that QAOA imitates QA [16]. Using variational parameters to rotate the energy particles and increasing the p parameter to decide the depth of the quantum circuit, should better approximate the annealing process. The problem to be solved with quantum computers are formalized with a Quadratic Unconstrained Binary Optimization (QUBO) based formulation, generally expressed in this mathematical way:

$$\text{minimize/maximize } y = x^t Q x$$

where x is a vector of binary decision variables and Q is a square matrix of constants, that can be symmetric or upper-triangular form [17]. A specific QUBO will be formulated for this thesis and presented in the next chapter.

³Optimization Performance of QA and QAOA - D-Wave White Paper (Accessed on: 15-01-26)

⁴IBM Quantum Computing (Accessed on: 15-01-2026)

2.4.2 Quantum Algorithms for RTSO

Different quantum algorithms have emerged that try to solve these problems. Other than choosing the base algorithm to use, depending on the related paradigm, the various proposed algorithms are different each from another by the decomposition techniques used to encode the input on a limited qubit number. Wang et al. proposed BootQA [18] such that QA can be used and adapted to solve test optimization, in particular for TSM. The original problem has been decomposed in smaller problems using bootstrap sampling. The results of this study showed how QA performs similar to Simulated Annealing (SA) and outperforms other quantum based optimization processes. QA has been used also for TCS. Trovato et al. proposed SelectQA [19] with a double type of Quadratic Unconstrained Binary Optimization (QUBO) formulation, to be able to compare this approach with multi-objective DIV-GA and single-objective Boot-QA. The decomposition of the problem, relies on the parallel run of different solvers by D-Wave Hybrid Solver Service (HSS). The study showed how SelectQA is the most effective and BootQA is the more efficient thanks to the dedicated decomposition technique, but the percentage of non-dominated solutions found by DIV-GA remains unbeaten. Wang et al. proposed another algorithm called IGDDec-QAOA [3], an impact guided decomposition of the problem to solve RTSO selecting test cases by how much they impact the improvement of the solution. This approach outperformed the baseline DIV-QAOA [3] where the algorithm does a non-impact guided decomposition and is on par with Genetic Algorithms solutions. Trovato et al. also investigate in a preliminary study the usage of QAOA, proposing QAOA-TCS [4] and decomposing the problem with an approach based on clustering, to preserve similarity and relationships in the sub test suites. The results showed an higher effectiveness than DIV-GA and SelectQA, but a similar efficiency with SelectQA. This would open to future comparison of multiples quantum approaches, but due to the D-Wave machines not longer available for free, QA based algorithms cannot be used at this time. For this reason, in this thesis the quantum algorithms used are only the QAOA based ones.

2.5 Related Work

In the context of performance regression testing and to the best of our knowledge, there is yet no study in literature related to the use of TCS in this specific field of software testing. However, Laaber et al. [20] investigated the applicability of TCP techniques to microbenchmarking suites. The applied TCP technique is the traditional one proposed by Rothermel et al. [21], that consists in a single-objective greedy algorithm based on total or additional coverage by the ranked microbenchmarks. The technique was also extended with a second ranking: the benchmarks affected by code changes were ranked and executed before those unaffected. The coverage was computed at the method-level, in a static way analyzing the call graph with WALA⁵ considering the benchmark granularity only at the method level, and in a dynamic way through JaCoCo⁶ considering the benchmark granularity at the parameter level, in a way that the same benchmark method can have multiple instances but different parameter values for state variables. The results showed how for the effectiveness, measured with Average Percentage of Fault Detected on Performance (AFPD-P) and the percentage of benchmark executed to detect the 3 largest performance changes (Top-3), the techniques using total coverage for greedy algorithm, outperformed the additional coverage, and dynamic-coverage enabled more effective techniques compared to static-coverage. For the efficiency, measured on the time overhead introduced by the specific TCP technique, the technique with dynamic coverage achieved 11% of overhead and the static one 4%. The two percentages are both considered acceptable and the best technique depends on the context where microbenchmarks are executed. The study leaves room for future research opportunities, with the aim to generalize the use of TCP techniques for performance testing that is now tailored for specific types of software, to explore black-box based TCP techniques, to involve human feedback loop in an experiment to decide which technique is better to use, to combine TCP and TCS techniques in a CI/CD environment. This thesis differs from the just presented study and wants to make a new contribution on various aspects:

⁵WALA - Git repository (Accessed on: 19-01-26)

⁶JaCoCo - Project Website (Accessed on: 19-01-26)

- **Different RTSO technique:** Instead of TCP, TCS related techniques are used. As stated in the future research opportunities of the related works, there is no approach that currently uses TCS techniques. Furthermore exploring TCS techniques, could create the possibility to combine the two techniques and fill the future research opportunities;
- **Variety of algorithms used:** In addition to an Additional Greedy algorithm, other types of algorithms are used, ranging from meta-heuristics algorithms such as DIV-GA to quantum algorithms that differ on decomposition techniques, such as an impact guided one with IGDec-QAOA and a clustering based one with QAOA-TCS. These four algorithms cover the entire State Of The Art for RTSO in functional testing;
- **Different level of code coverage:** Instead of method-level coverage, a statement-level coverage is computed that will return more precise results on which specific lines of code have performance issues;
- **Exploitation of JU2JMH to generate microbenchmarks:** One of the approaches proposed on this thesis, offers also the possibility to consider reusing an existing unit tests suite, with the aim of further reducing testing times and quickly creating a corresponding microbenchmarks suite using dedicated conversion tools. When there are no existing microbenchmarks suite, JU2JMH[6] is going to be used to convert the unit tests suite in a microbenchmarks one;
- **Generation of unit tests and microbenchmarks:** This thesis also explore the possibility to leverage the use of Large Language Models (LLMs) through a Prompt Engineering and Ablation Study process, to understand if it is possible to rely on the generation of suites for unit tests and microbenchmarks, further enhancing the reduction of overall testing activities times.

These reasons are explored in greater depth in the following chapters.

An Upstream and a Downstream Method to Create and Select Microbenchmarks

In this chapter, the method is proposed, and all the steps to retrieve the data, adapt the algorithms and the formulation, generate new test cases are detailed.

3.1 Method Overview

In order to create and select a subset of microbenchmarks, a new method is proposed that involves benchmarks run, testing metrics, algorithms execution. The method consists of two approaches that differ in how benchmarks are created and where test metrics are retrieved from.

3.1.1 Upstream Method Approach

The Upstream Method Approach aims to reuse the original functional test suites, such as unit tests classes built with JUnit, to select a subset of it and create a new microbenchmarks suite able to cover the lines of code affected by performance regressions. This approach is structured in various steps, that are presented as follow with the support of a building blocks figure:

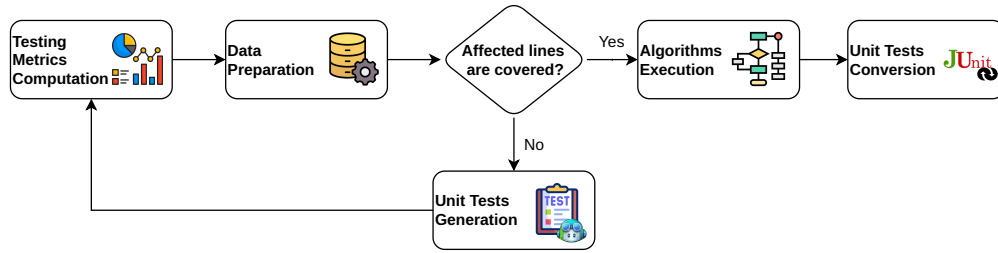


Figure 3.1: Upstream Method Approach - Building Blocks

1. **Testing Metrics Computation:** The existing unit tests are executed to retrieve coverage data and execution costs data;
2. **Data Preparation:** The raw testing metrics data are prepared in a convenient and more readable format;
3. **Unit Tests Generation:** If the prepared data shows that some affected lines are not covered, a unit tests generation campaign is conducted and the previous steps are executed again;
4. **Algorithms Execution:** The chosen algorithm are executed on the prepared data, to obtain as an output the subset of unit tests;
5. **JUnit Tests Conversion:** The selected unit tests are converted to microbenchmarks, thanks to *ju2jmh*[6].

3.1.2 Downstream Method Approach

The Downstream Approach aims to firstly create the microbenchmarks suite, such as JMH classes, and then select a subset of it able to cover the lines of code affected by performance regressions. This approach is structured in various steps, that are presented as follow with the support of a building blocks figure:

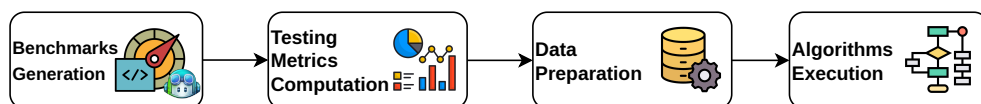


Figure 3.2: Downstream Method Approach - Building Blocks

1. **Benchmarks Generation:** A microbenchmark generation campaign is conducted to cover the affected lines and to provide a general coverage of the target class/package which they belong;
2. **Testing Metrics Computation:** The created microbenchmark are executed to retrieve coverage data and execution costs data;
3. **Data Preparation:** The raw testing metrics data are prepared in a convenient and more readable format;
4. **Algorithms Execution:** The chosen algorithm are executed on the prepared data, to obtain as an output the subset of microbenchmark.

3.2 Computing Testing Metrics

With all the suites ready to be executed, the approaches need to collect the data that will be then elaborated as the input for the algorithms execution.

3.2.1 Statement Coverage

JaCoCo¹ is a free code coverage library that offers a set of API and plugins for various build systems, such as a Maven². JaCoCo works through a Java Agent that uses Instrumentation API³, to enrich the byte-code and gather the necessary data for the coverage report. Thanks to JaCoCo API, Java Management Extensions (JMX)⁴ and through a custom built JUnit4 RunListener⁵ or JMH InternalProfiler⁶, it is possible to intercept the end of an execution of a single unit test or microbenchmark, to retrieve the byte-code stream of the specific computed coverage and redirect it to an .exec file, then used by the plugin to create a coverage report. In this way, the default behavior of JaCoCo is modified. Instead of having a single report for all the analyzed module, for each unit test or microbenchmark there is a dedicated report from which to extract

¹JaCoCo Java Code Coverage Library - Git repository (Accessed on: 20-01-2026)

²JaCoCo - Maven Plugin (Accessed on: 20-01-2026)

³Instrumentation API - JavaDoc (Accessed on: 20-01-2026)

⁴Java Management Extensions - JavaDoc (Accessed on: 20-01-26)

⁵JUnit4 RunListener - JavaDoc (Accessed on 20-01-26)

⁶JMH InternalProfiler - JavaDoc (Accessed on 20-01-26)

coverage information, line by line. In order to apply this process, the system has to be built with the execution of the tests. While for unit tests, there are no issue with running this setup, for microbenchmark a "simple-run" is necessary to rapidly end the execution of the microbenchmark and retrieve the coverage data. This simple run requires the JMH configuration for a single benchmarking process (`-f 1`) with one measurement iteration (`-i 1`) and no warmup phase, in the single shot mode (`-bm ss`) and output time in milliseconds (`-tu ms`). This configuration is sufficient to execute a single run of the benchmarks that will trigger the InternalProfiler and call the JaCoCo API for retrieving the coverage data.

3.2.2 Execution Costs

Surefire Plugin⁷ is used for executing the unit tests of the system. It generates reports in two different file formats (XML and TXT). The plugin also includes the execution times, in milliseconds, of all the successfully completed unit tests. To get reliable results in terms of execution time of microbenchmarks, JMH parameters configuration require a much more careful analysis. In particular about the warmup phase, the number of iterations are typically decided by developer expertise and not by previous studies in the field. In order to avoid bias no warmup iterations are executed. As stated by Traini et al. [11], an adequate mitigation to no warmups, is to use an high number of forks (e.g. 10) and measurement iterations. For these reasons, a more careful run requires the JMH configuration for a ten benchmarking processes (`-f 10`) with three thousand measurement iterations (`-i 3000`) and no warmup phase. The benchmark mode and output time unit remains the same as the simple run. The results are stored in a JSON file format (`-rff results.JSON -rf JSON`). To avoid the same sequential execution during the ten forks, microbenchmark are randomly chosen to avoid any other further optimization of the compiler. The configuration is applied together with the setup of the execution environment for the microbenchmark to remove most variability, coherently with Traini et al. [11]. It is recommended to disable CPU Boost (Intel Turbo Boost or AMD Precision Boost) and Multi Threading (Intel Hyper Threading or AMD Simultaneous Multi Threading),

⁷[Maven Surefire Plugin - Web Page](#) (Accessed on 20-01-26)

disable Address Space Layout Randomization (ASLR), set JVM heap memory to 8 GB and disable any Unix daemon that is not considered strictly necessary. All the JMH processes are assigned with the highest scheduling priority. In this way, reliable measurements of microbenchmark are collected.

3.3 Data Preparation

Once all the data for statement coverage and execution costs are collected, even if they are already structured and in a standardized format, such as XML and JSON, they can be considered raw data because they have to be processed again in the form of algorithms input. JaCoCo stores the coverage information in XML files, for each package and source file analyzed. A snippet of the source file coverage information, is showed as follows:

```
<sourcefile name="RecordBuilderBase.java">
  <line nr="106" mi="0" ci="2" mb="0" cb="0"/>
  <line nr="109" mi="0" ci="3" mb="0" cb="0"/>
  <line nr="110" mi="0" ci="3" mb="0" cb="0"/>
  <line nr="165" mi="7" ci="0" mb="2" cb="0"/>
  <line nr="167" mi="6" ci="0" mb="0" cb="0"/>
  <counter type="INSTRUCTION" missed="190" covered="22"/>
  <counter type="BRANCH" missed="24" covered="4"/>
  <counter type="LINE" missed="43" covered="8"/>
  <counter type="COMPLEXITY" missed="23" covered="3"/>
  <counter type="METHOD" missed="10" covered="2"/>
  <counter type="CLASS" missed="0" covered="1"/>
</sourcefile>
```

Figure 3.3: Source file coverage codified in an XML element

For each line, there are specific attributes describing if an instruction or a branch is covered/missed. A line is considered covered if the missed instructions attribute equals to zero (**mi="0"**). Starting from the XMLs, a coverage matrix is represented as a key-value pairs, where a key represents the unit test or microbenchmark name and the value represent the list of covered lines, uniquely identified by the Full Qualified Name (FQN) of the class and the line number, as shown in the following snippet:

```

{
  "org.apache.avro.data.RecordBuilderBaseTest.testIsValidValueWithNullField":
  [ "org.apache.avro.data.RecordBuilderBase{105;}",
    "org.apache.avro.data.RecordBuilderBase{109;}",
    "org.apache.avro.data.RecordBuilderBase{110;}",
    "org.apache.avro.data.RecordBuilderBase{113;}",
    "org.apache.avro.data.RecordBuilderBase{114;}" ],
  "org.apache.avro.data.RecordBuilderBaseTest.testIsValidValueWithPrimitives":
  [ "org.apache.avro.data.RecordBuilderBase{32;}",
    "org.apache.avro.data.RecordBuilderBase{105;}",
    "org.apache.avro.data.RecordBuilderBase{106;}",
    "org.apache.avro.data.RecordBuilderBase{109;}",
    "org.apache.avro.data.RecordBuilderBase{110;}",
    "org.apache.avro.data.RecordBuilderBase{113;}",
    "org.apache.avro.data.RecordBuilderBase{118;}",
    "org.apache.avro.data.RecordBuilderBase{127;}" ] ,
  "org.apache.avro.data.RecordBuilderBaseTest.testIsValidValueWithUnion":
  [ "org.apache.avro.data.RecordBuilderBase{105;}",
    "org.apache.avro.data.RecordBuilderBase{106;}",
    "org.apache.avro.data.RecordBuilderBase{109;}",
    "org.apache.avro.data.RecordBuilderBase{110;}",
    "org.apache.avro.data.RecordBuilderBase{113;}",
    "org.apache.avro.data.RecordBuilderBase{118;}",
    "org.apache.avro.data.RecordBuilderBase{119;}",
    "org.apache.avro.data.RecordBuilderBase{120;}",
    "org.apache.avro.data.RecordBuilderBase{121;}",
    "org.apache.avro.data.RecordBuilderBase{123;}",
    "org.apache.avro.data.RecordBuilderBase{127;}" ]
}

```

Figure 3.4: Coverage matrix representation in a JSON file format

Surefire stores the execution times in an XML file, as shown in the following:

```

<testcase name="testIsValidValueWithPrimitives"
classname="org.apache.avro.data.RecordBuilderBaseTest" time="0.081"/>
<testcase name="testIsValidValueWithUnion"
classname="org.apache.avro.data.RecordBuilderBaseTest" time="0.001"/>
<testcase name="testIsValidValueWithNullField"
classname="org.apache.avro.data.RecordBuilderBaseTest" time="0"/>

```

Figure 3.5: Unit tests execution times in an XML file format

JMH stores the execution times in a JSON file. The primaryMetric object contains the measurements data, as shown in the following:

```
"primaryMetric" : {
  "score" : 0.0034437740000000105,
  "scoreError" : 9.693707044404233E-5,
  "scoreConfidence" : [
    0.0033468369295559684,
    0.0035407110704440527
  ],
  "scorePercentiles" : {
    "0.0" : 6.92E-4,
    "50.0" : 0.001483,
    "90.0" : 0.007805,
    "95.0" : 0.011962,
    "99.0" : 0.021489799999999958,
    "99.9" : 0.054359635000000812,
    "99.99" : 0.11556601409995318,
    "99.999" : 0.301545,
    "99.9999" : 0.301545,
    "100.0" : 0.301545
  },
  "scoreUnit" : "ms/op",
  "rawData" : [
    [
      0.051126,
    ]
  ]
}
```

Figure 3.6: Microbenchmark execution times in a JSON file format

Execution times are retrieved to create a time matrix represented as a key-value pairs, where a key represents the name of a unit test or a microbenchmark and the value represents the execution time, as shown in the following JSON snippet:

```
{
  "org.apache.avro.data.RecordBuilderBaseTest.testIsValidValueWithNullField": "0.001",
  "org.apache.avro.data.RecordBuilderBaseTest.testIsValidValueWithPrimitives": "0.081",
  "org.apache.avro.data.RecordBuilderBaseTest.testIsValidValueWithUnion": "0.001",
}
```

Figure 3.7: Time matrix representation in a JSON file format

The two matrix representations give an easy and more manipulable format, to apply other transformations and obtain the final input files for each algorithms. For Additional Greedy and QAOA-TCS, the matrix are transformed, converting the unit test or microbenchmark name and the line of code number, in mapped numerical identifiers, to avoid that the algorithms use string identifiers during the computation and reducing the algorithm execution time. QAOA-TCS also need other two files, one reversing the coverage matrix representation to obtain the list of unit test or microbenchmark that cover a specific line of code and the other containing the total number of lines of code for the analyzed module of the system, computed with cloc⁸. For DIV-GA, the coverage matrix and time matrix previously mapped with numerical identifiers, are converted to a TXT file format. For IGDec-QAOA, a single CSV file is used, that contains three columns, each one with the unit test or microbenchmark name, execution time and statement coverage ratio.

3.4 Generating Unit Tests and Microbenchmarks

A tested system may lack some crucial unit tests, not achieving coverage on all the lines affected by performance issues. Given the limited time to gather knowledge on the system and to manually write new tests, it has been decided to involve the use of Large Language Models (LLMs) to automatically write the tests thanks to a detailed prompt. This activity has been carried out also for microbenchmark, where there were no starting suites that have been generated for the packages inside the identified module, that contains the classes with the affected lines to cover. In this way a general coverage for the packages has been provided, in order to recreate a scenario that could resemble a real case. The activity is divided into two phases:

- **3.4.1 - Prompt Engineering:** In this phase, the prompt is built through the use of meta-prompting and prompt-patterns, to be provided as input to the model;
- **3.4.2 - Ablation Study:** In this phase, the final prompt is evaluated in its variants to understand which one is the best in generating a valid output.

⁸Cloc - Git repository (Accessed on: 22-01-2026)

3.4.1 Prompt Engineering

Prompt engineering is the set of activities that design, test and optimize prompts input for LLMs [22]. In order to better construct the prompt, two patterns are used to schematize the interaction:

- **Persona** [23]: assigning an LLM a role to play when generating output;
- **Few-Shot** [24]: including examples of the desired output inside the prompt.

The initial prompt is obtained through a Meta-Prompt, a prompt that generates prompts [25]. Two meta-prompts, one for microbenchmark and one for unit tests, are realized and are showed as follow:

```
Act as a prompt engineer.

Generate the best possible prompt to use for creating a JUnit Class.

The prompt should specify:
- An "Act as a" incipit defining the role to assume
- The 4.13 version of JUnit
- The name of the production class code that has to be tested
- The specific class lines to cover
- To define a naming convention that should start with the "Test" prefix, followed by the
production class name and a suffix like "_1, _2, _A, _B" if a test class already exists
with that name
- An example of a simple test class

Write this prompt in a convenient format that can be easily reusable and parameterized for
scripting and automation.
```

Figure 3.8: Meta-prompt to obtain the prompt for unit tests generation

```
Act as a prompt engineer.

Generate the best possible prompt to use for creating a Java Micro Benchmark (JMH) Class.

The prompt should specify:
- An "Act as a" incipit defining the role to assume
- The latest version of JMH available
- The name of the production class code that has to be benchmarked
- The specific class lines to cover
- To avoid using test dependencies and mockups
- To use only legal imports in the scope of the project
- To avoid writing microbenchmark for combined use of methods
- To define a naming convention that should start with the production class name, followed
by "BenchmarkLLM" suffix
- To use only one inner state class to instantiate any type of dependency for the code
- An example of a simple micro benchmark class

Write this prompt in a convenient format that can be easily reusable and parameterized for
scripting and automation.
```

Figure 3.9: Meta-prompt to obtain the prompt for microbenchmarks generation

All the activities related to LLMs were done through the different Copilot versions: (i) the Web⁹ one, more suitable for conversation and used for generating the initial prompt through the specific meta-prompt, (ii) the CLI¹⁰ one, more suitable for scripting and automation and used for generating the unit tests and microbenchmark, (iii) the IDE Plugin¹¹, more suitable for troubleshooting during coding and used as a fallback. The model used through Copilot Web and CLI is Claude Sonnet 4.5¹². If the model had been unable to produce a valid output, the default model (GPT-5 mini¹³) through Copilot IDE Plugin would have been used as a fallback with a double interaction, using a simple and direct prompt to generate a unit test suite (e.g. *"Write a test class that covers the alter_table method and the lines from 565 to 575"*) and then to convert it (e.g. *"Create a JMH benchmark class converting this class"*).

⁹GitHub Copilot CLI - Website Page (Accessed on: 17-01-26)

¹⁰Github Copilot CLI - Website Page (Accessed on: 17-01-26)

¹¹Github Copilot IDE Plugin for IntelliJ - Website Page (Accessed on: 17-01-26)

¹²Anthropic Website - Claude Sonnet 4.5 Page (Accessed on 17-01-26)

¹³OpenAI Website - GPT-5 mini Page (Accessed on: 17-01-2026)

3.4.2 Ablation Study

An ablation study has been conducted to understand how these patterns affects the LLMs output, obtaining four versions of the prompt, based on the different ablation applied:

- **Ablation 0:** The prompt includes all the chosen prompt patterns;
- **Ablation 1:** The prompt keeps the persona pattern but removes few-shot;
- **Ablation 2:** The prompt removes the persona pattern but keeps few-shot;
- **Ablation 3:** The prompt removes all the chosen prompt patterns.

For each ablation, the prompt has been used for a five times interaction with the model to generate five different suites. The chosen ablation is the one that produces most unit tests or microbenchmark that are able to compile and can be executed. The table summarizes the numbers obtained:

Table 3.1: Generated unit tests and microbenchmarks for each ablation

	Unit Tests (#)	Microbenchmarks (#)
Ablation 0	113	23
Ablation 1	114	29
Ablation 2	107	33
Ablation 3	211	35

In this case, **Ablation 3** has been chosen.

3.5 Employed Algorithms

The RTSO algorithms considered State Of The Art (SOTA) are used for this thesis. A refactoring campaign is conducted, in order to adapt the algorithms coherently with the new QUBO formulation. The chosen algorithms are:

- **Additional Greedy:** The algorithm proposed by Yoo and Harman[1], creates a subset of the original test suite, selecting a test case to insert in the subset by how "convenient" is to add it, given the additional objectives values that contribute to the solution;

- **DIV-GA:** The algorithm proposed by Panichella et al.[2], creates a subset of the original test suite, selecting a test case to insert in the subset by the best fitness value, based on a multi-objective function computed at every iteration of the genetic routine;
- **IGDec-QAOA:** The algorithm proposed by Wang et al.[3], creates a subset of the original test suite, selecting a test case to insert in the subset by how it "impacts" the objective function values;
- **QAOA-TCS:** The algorithm proposed by Trovato et al.[4], creates a subset of the original test suite, selecting a test case to insert in the subset from the created clusters, by a similarity function.

The algorithms are implemented with different programming languages and specific third-party libraries, so a setup is needed for each one to be correctly executed. An Additional Greddy implementation is available in Python and uses the standard library, not requiring any additional dependencies. DIV-GA requires an installation of MATLAB R2025a¹⁴ and Global Optimization Toolbox¹⁵. The quantum algorithms requires Qiskit¹⁶ and Python 3.10.19¹⁷, in order to use the specific noise-simulator, called FakeBrisbane. A detailed list of dependencies for the quantum algorithms, is available inside the replication package¹⁸.

3.5.1 Adapting QUBO

Before executing the algorithms with the retrieved data, the problem is formalized with the formulation of a QUBO, coherent with Trovato et al. [19]. The QUBO contains linear terms representing the linear constraint on each test case and the quadratic constraint representing interaction between test cases, such as overlapping of statement coverage. The linear constraint has the goal to minimize the overall

¹⁴Matlab R2025a - Release Notes (Accessed on: 23-01-2026)

¹⁵Matlab Global Optimization Toolbox - Web Page (Accessed on: 23-01-2026))

¹⁶Qiskit - Web Page (Accessed on: 23-01-2026)

¹⁷Python 3.10.19 - Release Notes (Accessed on: 23-01-2026)

¹⁸Replication Package - Git repository (Accessed on: 01-02-2026)

execution costs of the subset of unit tests or microbenchmark, formalized as follows:

$$\sum_{i=1}^{|\Gamma|} [x_i \cdot cost(\tau_i)]$$

Where Γ stands for the starting suite, x_i stands for binary variable that specifies if the i -th test or benchmark is selected to be part of the solution, $cost(\tau_i)$ stands for the normalized execution cost of the i -th test or benchmark.

The quadratic constraint has the goal to maximize the statement coverage of the subset, formalized as follows:

$$P \cdot \sum_k \left(\sum_{i \in T_k} (x_i - 1)^2 \right)$$

Where P stands for the multiplied penalty constant [17], guiding the optimizer to search acceptable solutions, the ones that do not incur penalties. The value of P is set to a value slightly higher than the maximum value achievable by the objective function. In this way the penalty aligns with the application domain and significantly influences the solution process [26]. The k stands for a statement to cover, T_k is the list of tests or benchmarks that cover the k -th statement, x_i stands for binary variable that specifies if the i -th test or benchmark is selected to be part of the solution. The constraints make up the complete function:

$$\sum_{i=1}^{|\Gamma|} [x_i \cdot cost(\tau_i)] + P \cdot \sum_k \left(\sum_{i \in T_k} (x_i - 1)^2 \right)$$

The function is then simplified and completely linearized:

$$\sum_{i=1}^{|\Gamma|} [x_i \cdot cost(\tau_i)] + \sum_k \sum_{i,j \in T_k} (-Px_i + 2Px_i x_j)$$

This function is codified inside the different algorithms that will be executed.

CHAPTER 4

Empirical Study

In this chapter, the results from the statistical analysis are showed and are discussed to compare the approaches in a quantitative way, answering the two Research Questions.

4.1 Goal and Research Questions

The goal of this thesis is to explore the use of TCS techniques in the context of microbenchmarking, formalized through the formulation of two Research Questions:

Q RQ₁. *Which is the best algorithm for selecting a subset of microbenchmarks?*

Q RQ₂. *Which is the best method approach for creating and selecting microbenchmarks?*

The first Research Question aims to evaluate the best algorithm for each method approach, breaking down the research question:

Q RQ_{1.a}. *Which algorithm is able of selecting the most effective subset of microbenchmarks?*

Q RQ_{1.b}. *Which algorithm is able of selecting the most efficient subset of microbenchmarks?*

The effectiveness is measured in terms of achieved statement coverage of the selected subset of microbenchmarks. The efficiency is measured in terms of total execution costs of the selected subset of microbenchmarks. The second Research Question aims

to compare the upstream and downstream method approaches to establish which one is the best. In order to apply the method approaches and answer the Research Questions, two systems with specific characteristics are analyzed.

4.2 Systems Overview

The systems undergone a previous mining study that discovered different code fragments affected by performance issues. Starting from the git repository, the systems have been analyzed by Campos et al. [27] on two commits, namely the commit where the issues have been identified, **prior** to the fix, and the commit where the issues have been removed, **posterior** to the fix. For each commit, the range of lines of code affected by performance issues are listed together with the class they are from.

Avro¹ is a data serialization system. It is considered the leading serialization format for record data and first choice for streaming data pipelines². The **avro** module is the identified one that contains the lines of code with discovered performance issues. The pre-fix³ and post-fix⁴ commits are uniquely identified by their hash:

Table 4.1: Lines of code with known performance issues - Avro

Pre-fix commit - cd91695168f9edaac8c1f0140f4a023258f265d4	
Class Full Qualified Name	Lines of code
org.apache.avro.data.RecordBuilderBase	76, 77
org.apache.avro.util.Utf8	64, 65, 116–118
Post-fix commit - ff5c4d6db9af3edd915a2176a627d51eb3478da3	
Class Full Qualified Name	Lines of code
org.apache.avro.data.RecordBuilderBase	76
org.apache.avro.util.Utf8	65, 116

Hive⁵ is a distributed, fault-tolerant data warehouse system that enables analytics at a massive scale and facilitates reading, writing, and managing petabytes of data. The **standalone-metastore-common** module is the identified one that contains the

¹Apache Avro - Project Website (Accessed on: 16-01-26)

²Apache Avro - Git repository (Accessed on: 16-01-26)

³Pre-fix commit for Avro (Accessed on: 16-01-2026)

⁴Post-fix commit for Avro (Accessed on: 16-01-2026)

⁵Apache Hive - Project Website (Accessed on: 16-01-26)

lines of code with discovered performance issues. The pre-fix⁶ and post-fix⁷ commits are uniquely identified by their hash:

Table 4.2: Lines of code with known performance issues - Hive

Pre-fix commit - 186fb0d85af63b61bc10ba5372e35895754b1a6a	
Class Full Qualified Name	Lines of code
org.apache.hadoop.hive.metastore.HiveMetaStoreClient	565-572, 575
org.apache.hadoop.hive.metastore.utils.MetaStoreUtils	802-813, 815-824
Post-fix commit - 8863468898c9242fc334b9c93bf79e249e53842d	
Class Full Qualified Name	Lines of code
org.apache.hadoop.hive.metastore.HiveMetaStoreClient	567-581
org.apache.hadoop.hive.metastore.utils.MetaStoreUtils	802-825

Starting from these two pair of commits, the systems are cloned and a checkout is done on the specific commit, to understand how to build the system and to verify how many unit tests and microbenchmarks were already written by the developers. If necessary, new test cases will be generated as described in section 3.4.

4.3 Experiment Configuration

To apply the method approaches, the algorithms execution have to be configured with specific parameters values. Additional Greedy does not require any hyperparameter to set and due to its deterministic nature, a single execution is sufficient. DIV-GA hyperparameters are the same as the one used by Panichella et al. [2] and the algorithm is executed ten times. For the quantum algorithms, the **depth** of the quantum circuit is set at **1**, the number of **qubits** at **7**, the same used by Wang et al. [3] Trovato et al. [4], and the algorithms are also executed ten times. All the algorithms were executed on a machine running Linux Mint 22.2⁸ (based on Ubuntu 24.04 LTS) with an AMD Ryzen 7 5700G (8 cores) and 32 GB of RAM.

⁶Pre-fix commit for Hive (Accessed on: 16-01-26)

⁷Post-fix commit for Hive (Accessed on: 16-01-26)

⁸Linux Mint - Project Website (Accessed on: 23-01-2026)

4.4 Evaluation Metrics

After the algorithms execution, the outputs are compared to retrieve the results. The output of an algorithm represents a pareto front, a set of non dominated solutions. Given that the algorithms are executed ten times on the analyzed system data and on the pre-fix and post-fix commit, there are in total 20 pareto fronts for each algorithm. For each pareto front, the mean and the standard deviation are computed, in terms of achieved statement coverage on the lines of code related to performance issues for the effectiveness and in terms of execution times of the unit tests or microbenchmarks that make up the solution for the efficiency. The 20 mean and standard deviation values, one for each pareto front, are then used to conduct some statistical tests and compare the algorithms. The first conducted test is the Shapiro-Wilk [28], that checks if a sample follows the normal distribution. Depending on how this test goes, a post-hoc analysis is conducted. The post-hoc analysis will check if there are any pair combinations that have a significant statistical difference, then check which specific pair combinations differs, to quantify the difference. In this case all the sample are not following a normal distribution, so Kruskal-Wallis H [29], Dunn with Bonferroni correction [30] tests are executed. The tests are considered statistical significant when $p\text{-value} < 0.05$. After the tests, Vargha-Delaney $\hat{A}$ [5] effect size is computed to obtain a measure of the differences. The process is repeated two times, one considering the pareto front objective values both in terms of effectiveness with the statement coverage and one in terms of efficacy with the execution costs. This evaluation process is consistent with that carried out by Panichella et al. [2] and Trovato et al. [19] [4].

4.5 RQ1 - Which is the best algorithm for selecting a subset of microbenchmarks?

RQ1.a - Which algorithm is able of selecting the most effective subset of microbenchmarks?

As showed in the Table 4.3, DIV-GA is the algorithm that achieves the highest statement coverage, on both analyzed systems and with both approaches.

Table 4.3: Mean and Standard Deviation for pareto fronts - statement coverage

Program	Approach	Algorithm	Mean	St. Dev.
Avro	Upstream	Additional Greedy	0.491	0.035
		DIV-GA	0.991	0.027
		IGDec-QAOA (noise)	0.325	0.038
		QAOA-TCS (noise)	0.406	0.151
	Downstream	Additional Greedy	0.658	0.020
		DIV-GA	1.000	0.000
		IGDec-QAOA (noise)	0.150	0.037
		QAOA-TCS (noise)	0.423	0.174
Hive	Upstream	Additional Greedy	0.622	0.032
		DIV-GA	0.641	0.117
		IGDec-QAOA (noise)	0.137	0.072
		QAOA-TCS (noise)	0.290	0.174
	Downstream	Additional Greedy	0.556	0.046
		DIV-GA	0.817	0.055
		IGDec-QAOA (noise)	0.234	0.084
		QAOA-TCS (noise)	0.620	0.134

For the data from the Avro system, DIV-GA is able to select suites with almost total coverage on a 99% for JUnit and 100% for JMH. QAOA-TCS is the best quantum algorithm selecting suites with a coverage ranging around 40% for both JUnit and JMH. The DIV-GA results are supported by statistical tests reported in Table 4.4. All the comparisons that favor DIV-GA have statistical differences and have showed a large magnitude, confirming the observed differences. QAOA-TCS is the best quantum algorithm, but it does not have a complete statistical significance, recording a non confirmed comparison for JUnit suites.

For the data from Hive system, DIV-GA is still unbeaten but with lower percentages,

4.5 – RQ1 - Which is the best algorithm for selecting a subset of microbenchmarks?

selecting suites with 64% of coverage on JUnit and 82% on JMH. QAOA-TCS remains the best quantum algorithm and is able to break the 60% coverage wall, with a 62% on selected JMH suites. DIV-GA results are partially supported by the statistical tests. The 2 percentage points difference between DIV-GA and Additional Greedy on JUnit suites, have not permitted to record a significant difference and comparison between the two for both suites, but DIV-GA is confirmed to have a statistical difference on the quantum algorithms with a large magnitude. QAOA-TCS achievements are again partially confirmed by statistical significance, with the only significant comparison on JMH suites. DIV-GA is confirmed again to have a statistical significant difference against all algorithms on JMH suites, with a large magnitude.

Table 4.4: Dunn’s test and $\hat{\Delta}$ effect size - statement coverage

Program	Approach	Comparison (higher coverage is better)	p-value	Adj. p-value	$\hat{\Delta}$
Avro	Upstream	DIV-GA > Add. Greedy	< 0.05	< 0.05	1 (L)
		DIV-GA > IGDec-QAOA (noise)	< 0.05	< 0.05	1 (L)
		DIV-GA > QAOA-TCS (noise)	< 0.05	< 0.05	1 (L)
		QAOA-TCS (noise) > IGDec-QAOA (noise)	0.03	0.18	-
	Downstream	DIV-GA > Add. Greedy	< 0.05	< 0.05	1 (L)
		DIV-GA > IGDec-QAOA (noise)	< 0.05	< 0.05	1 (L)
		DIV-GA > QAOA-TCS (noise)	< 0.05	< 0.05	1 (L)
		QAOA-TCS (noise) > IGDec-QAOA (noise)	< 0.05	< 0.05	0.953 (L)
Hive	Upstream	DIV-GA > Add. Greedy	0.82	1	-
		DIV-GA > IGDec-QAOA (noise)	< 0.05	< 0.05	1 (L)
		DIV-GA > QAOA-TCS (noise)	< 0.05	< 0.05	0.955 (L)
		QAOA-TCS (noise) > IGDec-QAOA (noise)	0.06	0.37	-
	Downstream	DIV-GA > Add. Greedy	< 0.05	< 0.05	1 (L)
		DIV-GA > IGDec-QAOA (noise)	< 0.05	< 0.05	1 (L)
		DIV-GA > QAOA-TCS (noise)	< 0.05	< 0.05	0.875 (L)
		QAOA-TCS (noise) > IGDec-QAOA (noise)	< 0.05	< 0.05	1 (L)

4.5 – RQ1 - Which is the best algorithm for selecting a subset of microbenchmarks?

RQ1.b - Which algorithm is able of selecting the most efficient subset of microbenchmarks?

As showed in the Table 4.5, quantum algorithms are battling with DIV-GA and are even surpassing it.

Table 4.5: Mean and Standard Deviation for pareto fronts - execution costs (in milliseconds)

Program	Approach	Algorithm	Mean	St. Dev.
Avro	Upstream	Additional Greedy	0.0035	0.0002
		DIV-GA	0.0010	2.6492
		IGDec-QAOA (noise)	0.0014	7.2017
		QAOA-TCS (noise)	0.0010	0.0001
	Downstream	Additional Greedy	0.0055	0.0007
		DIV-GA	0.0025	0.0007
		IGDec-QAOA (noise)	0.0019	0.0007
		QAOA-TCS (noise)	0.0016	0.0006
Hive	Upstream	Additional Greedy	0.0200	0.0007
		DIV-GA	0.0043	0.0001
		IGDec-QAOA (noise)	0.0016	0.0010
		QAOA-TCS (noise)	0.0041	0.0005
	Downstream	Additional Greedy	0.1109	0.0006
		DIV-GA	0.0158	0.0014
		IGDec-QAOA (noise)	0.0041	0.0019
		QAOA-TCS (noise)	0.0100	0.0020

For the data from the Avro system, DIV-GA is on par with QAOA-TCS for JUnit suites, but for JMH suites DIV-GA select the ones with 25ms, behind the two quantum algorithms, that respectly select the ones with 19ms at the second place for IGDec-QAOA and 16ms at first place for QAOA-TCS, stating the first time a quantum algorithm beat a traditional one. DIV-GA still remains the best traditional algorithm. The results are partially supported by the statistical tests reported in Table 4.6. Given no differences in terms of milliseconds, there is no significant difference between QAOA-TCS and DIV-GA on JUnit suites. The same goes for JMH suites, also between the two quantum ones. DIV-GA is confirmed as the best traditional one.

For the data from the Hive system, IGDec-QAOA selects better suites with 16ms on JUnit and also on JMH, IGDec-QAOA selects the fastest suites with 41ms. The results are almost all confirmed with statistical tests. IGDec-QAOA have significant

4.5 – RQ1 - Which is the best algorithm for selecting a subset of microbenchmarks?

differences with a large magnitude against the traditional algorithms, but not for QAOA-TCS, where there is no a valid comparison on JMH suites. DIV-GA confirms again to be the best traditional algorithm.

Table 4.6: Dunn’s test and $\hat{A}$ effect size - execution costs (in milliseconds)

Program	Approach	Comparison (lower costs is better)	p-value	Adj. p-value	$\hat{A}$
Avro	Upstream	DIV-GA $\leq$ Add. Greedy	< 0.05	< 0.05	1 (L)
		DIV-GA $\leq$ IGDdec-QAOA (noise)	< 0.05	< 0.05	1 (L)
		DIV-GA $\leq$ QAOA-TCS (noise)	0.48	1	-
		QAOA-TCS (noise) $\leq$ IGDdec-QAOA (noise)	< 0.05	< 0.05	0.987 (L)
	Downstream	QAOA-TCS $\leq$ Add. Greedy	< 0.05	< 0.05	1 (L)
		QAOA-TCS (noise) $\leq$ DIV-GA	0.01	0.11	-
		QAOA-TCS (noise) $\leq$ IGDdec-QAOA (noise)	0.43	0.10	-
		DIV-GA $\leq$ Add. Greedy	< 0.05	< 0.05	1 (L)
Hive	Upstream	IGDdec-QAOA (noise) $\leq$ Add. Greedy	< 0.05	< 0.05	1 (L)
		IGDdec-QAOA (noise) $\leq$ DIV-GA	< 0.05	< 0.05	0.950 (L)
		IGDdec-QAOA (noise) $\leq$ QAOA-TCS (noise)	< 0.05	< 0.05	0.930 (L)
		DIV-GA $\leq$ Add. Greedy	< 0.05	< 0.05	1 (L)
	Downstream	IGDdec-QAOA (noise) $\leq$ Add. Greedy	< 0.05	< 0.05	1 (L)
		IGDdec-QAOA (noise) $\leq$ DIV-GA	< 0.05	< 0.05	1 (L)
		IGDdec-QAOA (noise) $\leq$ QAOA-TCS (noise)	0.01	0.06	-
		DIV-GA $\leq$ Add. Greedy	< 0.05	< 0.05	1 (L)

✍ **Answer to RQ₁.** DIV-GA is the algorithm capable of selecting the most effective subset of microbenchmarks, for both upstream and downstream approach. The quantum algorithms are capable of selecting the most efficient subset of microbenchmarks on both approaches.

4.6 RQ2 - Which is the best approach for creating and selecting microbenchmarks?

Table 4.7: Dunn’s test and $\hat{A}$ effect size - extra comparisons for execution costs

Program	Approach	Comparison (lower costs is better)	p-value	Adj. p-value	$\hat{A}$
Avro	Upstream	QAOA-TCS (noise) $\leq$ DIV-GA	0.48	1	-
	Downstream	QAOA-TCS (noise) $\leq$ DIV-GA	< 0.05	0.11	-
Hive	Upstream	QAOA-TCS (noise) $\leq$ DIV-GA	0.24	1	-
	Downstream	QAOA-TCS (noise) $\leq$ DIV-GA	< 0.05	< 0.05	0.982 (L)

Having established that DIV-GA is the best algorithm for selecting solutions in terms of effectiveness for both approaches, and that Quantum Algorithms alternate in being the best algorithms for selecting solutions in terms of efficiency for both algorithms, it is possible to better observe the data to understand which may be the best configuration of algorithm and approach to use. Focusing on DIV-GA and the data relating to the efficiency of the selected solutions, it is clear that, even in terms of efficiency, DIV-GA is closer to quantum algorithms than to the other traditional algorithm (Additional Greedy). As shown in Table 4.7, DIV-GA is so close that, based on the data analyzed for the Avro system, the statistical comparisons made with QAOA-TCS are not considered significant, given the small differences in the results obtained. This behavior is also confirmed by the Hive data, where IGDDec-QAOA is the best algorithm with a remarkable advantage over QAOA-TCS, but DIV-GA continues to be very close to the second-best quantum algorithm. In addition, there is still difficulty to a proper access for using Quantum Algorithm in a real environment, given the low qubit and quantum machines availability, while DIV-GA stands out for the easy of use thanks to a well-known tool for optimization algorithms such as MATLAB and the simplicity to re-implement the algorithm with other programming languages, through various frameworks for meta-heuristics algorithm (e.g. *jMetal*, *pymoo*). Further consideration should also be given to the adequacy of the testing tools used. Although the execution times collected by JUnit are the lowest, these are linked to a single execution of the unit tests and therefore cannot be relied upon too heavily, in contrast to JMH, which is a tool specifically designed to collect multiple

4.6 – RQ2 - Which is the best approach for creating and selecting microbenchmarks?

execution times in a more robust and reliable manner. Finally, it is necessary to consider how the JaCoCo coverage tool is typically used. It is always associated with functional testing (unit testing) but has proven capable of collecting coverage data through other types of testing, such as microbenchmarks, thanks to a simpler and more dedicated execution.

✚ **Answer to RQ₂.** Based on the well balanced results obtained with DIV-GA and how easy it is to use this algorithm, in combination with the reliable measurements that can be done through microbenchmarking thanks to JMH, the downstream approach with DIV-GA is the best configuration compared to all the others analyzed.

CHAPTER 5

Discussion

In this chapter, the results are discussed in a more qualitative way, considering some factors and aspects that could have favored one approach instead of the other one.

Specifically crafted test suites. The systems were lacking some unit-test suites and entirely missing micro-benchmark ones, so a generation campaign has been conducted. Being JUnit suited for functional testing, most of the necessary test suites already existed in the systems and the study relied directly on them. However for JMH, since the systems did not have existing suites, dedicated ones were generated, focusing mainly on the affected lines and the class/package to which they belong.

Q Observation 1. The way the approaches relies on existing or specific generated test suites, may have favored the downstream approach over the upstream one.

Analyzed systems. A software system could be realized for different context and different aspects of the society. Some properties of the system, could favor an objective more than another. Considering that the two analyzed system are data-intensive applications, in this case an approach that prioritize the effectiveness, being able to specific detect lines affected by performance issues, should be favored sacrificing the efficiency and increasing the execution times of the performance testing activities, because data-processing systems could deal with critical context and expensive constraints, where optimizing execution times of the deployed system should be

a prioritized factor. In other cases, where a software is tested and deployed in less critical context, an approach that prioritize the efficiency, being able to select a suite with fastest execution times, should be favored even if this approach could miss some performance issues because the main goal could be different and more related to how fast new versions are delivered, relaxing other constraints.

Q Observation 2. There is a no absolute objective to prefer, but it all depends on the type of system and on the current necessity of the testing campaign.

Software cost optimization. While reducing cost for performance testing is important for reducing the allocated time for testing activities, on a broader view the goal is to optimize in general the costs to produce and maintain a software. Preferring techniques that leverage already existent suites, such as JUnit ones, could be a solid choice. Instead of selecting directly on the JUnit suites, with an unreliable collection of execution times data, a variant of the upstream approach could be an automatic conversion of all JUnit suites to JMH ones thanks to dedicated tool, such as JU2JMH [6], and then execute the algorithms, allocating smaller costs for micro-benchmarks. However, using these conversion tools return higher execution times, due to the application of wrapping techniques on top of already existent JUnit suites.

Q Observation 3. Reducing the costs for performance testing, could imply its complete automation to optimize the testing costs.

CHAPTER 6

Threats to Validity

In this chapter, the validity of the experiment is exposed to the relative threats, and it is described how the mitigations were applied

6.1 Construct Validity

There are two problems related to construct validity. The first one is the choice of the testing metrics used to evaluate the approaches, that are code coverage and execution costs. The second one is the assumption of the correctness of the used tools to retrieve the data related to statement coverage (JaCoCo) and execution times (Surefire Plugin and the JMH configuration). In particular for the execution times, Surefire is able to collect times for a single unit tests execution and with a low precision capability. The values of the test cases with zero-time (0.000) were replaced with a default value. Instead for JMH, being a tool directly designed for this task, there is a native support for multiple execution of microbenchmarks and a more precise time collection. These problems are mitigated thanks to the existent literature of RTSO for functional testing, where the two metrics are involved in the formulation of single and multi-objective algorithms [1] [2] [4] [19]. In addition the used tools to retrieve the data are well-known and open source, also used in the related work [20].

6.2 Internal Validity

The main problem related to internal validity is the non deterministic nature of algorithms and LLMs involved. This problem is mitigated using specific parameters that tune up the execution of the algorithms, such as the hyperparameter configuration adopted by Panichella et al. [2] for DIV-GA, the depth of the circuit and the problem size parameters for quantum algorithms, fixed as the same values of Wang et al. [3] and Trovato et al. [19] [4]. The penalty constant P of the QUBO formulation, is coherently added in the same way as done by Trovato et al. [19]. The non deterministic algorithms are also executed ten times to stabilize the obtained outputs, coherently with Trovato et al. [4] [19]. In addition, to obtain more reliable outputs from the LLMs, a Prompt Engineering and Ablation Study has been conducted to establish the best version of the prompt to be used for the generation of the suites.

6.3 External Validity

The main problem related to external validity, is the low number of systems analyzed. For this study only two systems (Avro and Hive) are analyzed on some specific modules. In addition, the performance issues are more diffused on the Hive system than on the Avro one. This problem is mitigated by how the two system were chosen, already undergone a previously mining study by Campos et al. [27], that identified performance issues inside the systems. The system have been analyzed on two commits, that represents two different versions of the system, the one with non-fixed issued and the one with the fixed. The two systems are also well-known and large-scale system, totally open source. In addition, this thesis represents a preliminary study in the field, that could be explored further in the future.

6.4 Conclusion Validity

The main problem related to conclusion validity is the significance of the results. This problem is mitigated by the conducted statistical tests. The results not confirmed by statistical tests, are still analyzed but are not considered valid and can be ignored.

CHAPTER 7

Conclusions

This thesis addressed the problem of RTSO in the field of Performance Testing. Two approaches are proposed, based on already existent and adapted TCS techniques. The two analyzed systems showed how traditional algorithms such as DIV-GA are still competitive solutions and balanced in terms of effectiveness/efficacy, with how easy is to use this type of algorithms and how simple is to integrate them inside a structured approach, in opposition of quantum ones that look promising in terms of efficacy but are still inadequate on effectiveness terms and in general have a limited ease of use in a real context. The results from the Research Questions, highlight how there is a better configuration of the algorithm and the approach, but that it all depends on the analyzed context, on the type of system and on how much time can be allocated for testing activities. The thesis opens up to possible future research opportunities. The study should be extended, analyzing other systems to better generalize the obtained results, also exploring the use of new algorithms variants, involving the ideal simulator of the quantum ones. The approaches could be then implemented inside a Continuous Integration and Continuous Delivery pipeline.

Bibliography

- [1] S. Yoo and M. Harman, “Pareto efficient multi-objective test case selection,” in *Proceedings of the 2007 international symposium on Software testing and analysis*, 2007, pp. 140–150. (Citato alle pagine 2, 11, 27 e 42)
- [2] A. Panichella, R. Oliveto, M. Di Penta, and A. De Lucia, “Improving multi-objective test case selection by injecting diversity in genetic algorithms,” *IEEE Transactions on Software Engineering*, vol. 41, no. 4, pp. 358–383, 2014. (Citato alle pagine 2, 11, 12, 28, 32, 33, 42 e 43)
- [3] X. Wang, S. Ali, T. Yue, and P. Arcaini, “Quantum approximate optimization algorithm for test case optimization,” *IEEE Transactions on Software Engineering*, 2024. (Citato alle pagine 2, 14, 28, 32 e 43)
- [4] A. Trovato, M. Beseda, and D. Di Nucci, “A preliminary investigation on the usage of quantum approximate optimization algorithms for test case selection,” *arXiv preprint arXiv:2504.18955*, 2025. (Citato alle pagine 2, 14, 28, 32, 33, 42 e 43)
- [5] A. Vargha and H. D. Delaney, “A critique and improvement of the cl common language effect size statistics of mcgraw and wong,” *Journal of Educational and Behavioral Statistics*, vol. 25, no. 2, pp. 101–132, 2000. (Citato alle pagine 3 e 33)
- [6] M. Jangali, Y. Tang, N. Alexandersson, P. Leitner, J. Yang, and W. Shang, “Automated generation and evaluation of jmh microbenchmark suites from unit

- tests,” *IEEE Transactions on Software Engineering*, vol. 49, no. 4, pp. 1704–1725, 2022. (Citato alle pagine 4, 16, 18 e 41)
- [7] S. Balsamo, A. Di Marco, P. Inverardi, and M. Simeoni, “Software performance: state of the art and perspectives,” 2002. (Citato a pagina 6)
- [8] D. Beyer, S. Löwe, and P. Wendler, “Reliable benchmarking: requirements and solutions,” *International Journal on Software Tools for Technology Transfer*, vol. 21, no. 1, pp. 1–29, 2019. (Citato a pagina 7)
- [9] C. Laaber and P. Leitner, “An evaluation of open-source software microbenchmark suites for continuous performance assessment,” in *Proceedings of the 15th international conference on mining software repositories*, 2018, pp. 119–130. (Citato a pagina 7)
- [10] C. Laaber, J. Scheuner, and P. Leitner, “Software microbenchmarking in the cloud. how bad is it really?” *Empirical Software Engineering*, vol. 24, no. 4, pp. 2469–2508, 2019. (Citato a pagina 7)
- [11] L. Traini, V. Cortellessa, D. Di Pompeo, and M. Tucci, “Towards effective assessment of steady state performance in java software: Are we there yet?” *Empirical Software Engineering*, vol. 28, no. 1, p. 13, 2023. (Citato alle pagine 8 e 20)
- [12] S. Yoo and M. Harman, “Regression testing minimization, selection and prioritization: a survey,” *Software testing, verification and reliability*, vol. 22, no. 2, pp. 67–120, 2012. (Citato a pagina 10)
- [13] G. Rothermel and M. J. Harrold, “A framework for evaluating regression test selection techniques,” in *Proceedings of the 16th international conference on Software engineering*, 1994, pp. 201–210. (Citato a pagina 11)
- [14] K. Deb, S. Agrawal, A. Pratap, and T. Meyarivan, “A fast elitist non-dominated sorting genetic algorithm for multi-objective optimization: Nsga-ii,” in *International conference on parallel problem solving from nature*. Springer, 2000, pp. 849–858. (Citato a pagina 11)

-
- [15] S. Kumari, S. Jindal, and A. Sharma, "Test case optimization using grey wolf algorithm," *Software Quality Journal*, vol. 33, no. 2, pp. 1–30, 2025. (Citato a pagina 12)
- [16] S. Yarkoni, E. Raponi, T. Bäck, and S. Schmitt, "Quantum annealing for industry applications: Introduction and review," *Reports on Progress in Physics*, vol. 85, no. 10, p. 104001, 2022. (Citato a pagina 13)
- [17] F. Glover, G. Kochenberger, R. Hennig, and Y. Du, "Quantum bridge analytics i: a tutorial on formulating and using qubo models," *Annals of Operations Research*, vol. 314, no. 1, pp. 141–183, 2022. (Citato alle pagine 13 e 29)
- [18] X. Wang, A. Muqet, T. Yue, S. Ali, and P. Arcaini, "Test case minimization with quantum annealers," *ACM Transactions on Software Engineering and Methodology*, vol. 34, no. 1, pp. 1–24, 2024. (Citato a pagina 14)
- [19] A. Trovato, M. De Stefano, F. Pecorelli, D. Di Nucci, and A. De Lucia, "Reformulating regression test suite optimization using quantum annealing-an empirical study," *International Journal on Software Tools for Technology Transfer*, vol. 26, no. 6, pp. 767–780, 2024. (Citato alle pagine 14, 28, 33, 42 e 43)
- [20] C. Laaber, H. C. Gall, and P. Leitner, "Applying test case prioritization to software microbenchmarks," *Empirical Software Engineering*, vol. 26, no. 6, p. 133, 2021. (Citato alle pagine 15 e 42)
- [21] G. Rothermel, R. H. Untch, C. Chu, and M. J. Harrold, "Test case prioritization: An empirical study," in *Proceedings IEEE International Conference on Software Maintenance-1999 (ICSM'99). 'Software Maintenance for Business Change' (Cat. No. 99CB36360)*. IEEE, 1999, pp. 179–188. (Citato a pagina 15)
- [22] S. Lemeš, "Prompt engineering," *Artificial intelligence in industry*, vol. 4, pp. 159–170, 2024. (Citato a pagina 25)
- [23] J. White, "A prompt pattern catalog to enhance prompt engineering with chatgpt," *arXiv preprint arXiv:2302.11382*, 2023. (Citato a pagina 25)

- [24] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Nee-lakantan, P. Shyam, G. Sastry, A. Askell *et al.*, “Language models are few-shot learners,” *Advances in neural information processing systems*, vol. 33, pp. 1877–1901, 2020. (Citato a pagina 25)
- [25] V. Klouda, “Meta-prompts for llm prompt optimization,” 2025. (Citato a pagina 25)
- [26] M. Ayodele, “Penalty weights in qubo formulations: Permutation problems,” in *European Conference on Evolutionary Computation in Combinatorial Optimization (Part of EvoStar)*. Springer, 2022, pp. 159–174. (Citato a pagina 29)
- [27] D. Campos, L. Martins, E. Guglielmi, M. Tucci, D. Di Pompeo, S. Scalabrino, V. Cortellessa, D. Di Nucci, and R. Oliveto, “Identifying and replicating code patterns driving performance regressions in software systems,” *arXiv preprint arXiv:2504.05851*, 2025. (Citato alle pagine 31 e 43)
- [28] S. S. Shapiro and M. B. Wilk, “An analysis of variance test for normality (complete samples),” *Biometrika*, vol. 52, no. 3-4, pp. 591–611, 1965. (Citato a pagina 33)
- [29] W. H. Kruskal and W. A. Wallis, “Use of ranks in one-criterion variance analysis,” *Journal of the American statistical Association*, vol. 47, no. 260, pp. 583–621, 1952. (Citato a pagina 33)
- [30] O. J. Dunn, “Multiple comparisons using rank sums,” *Technometrics*, vol. 6, no. 3, pp. 241–252, 1964. (Citato a pagina 33)